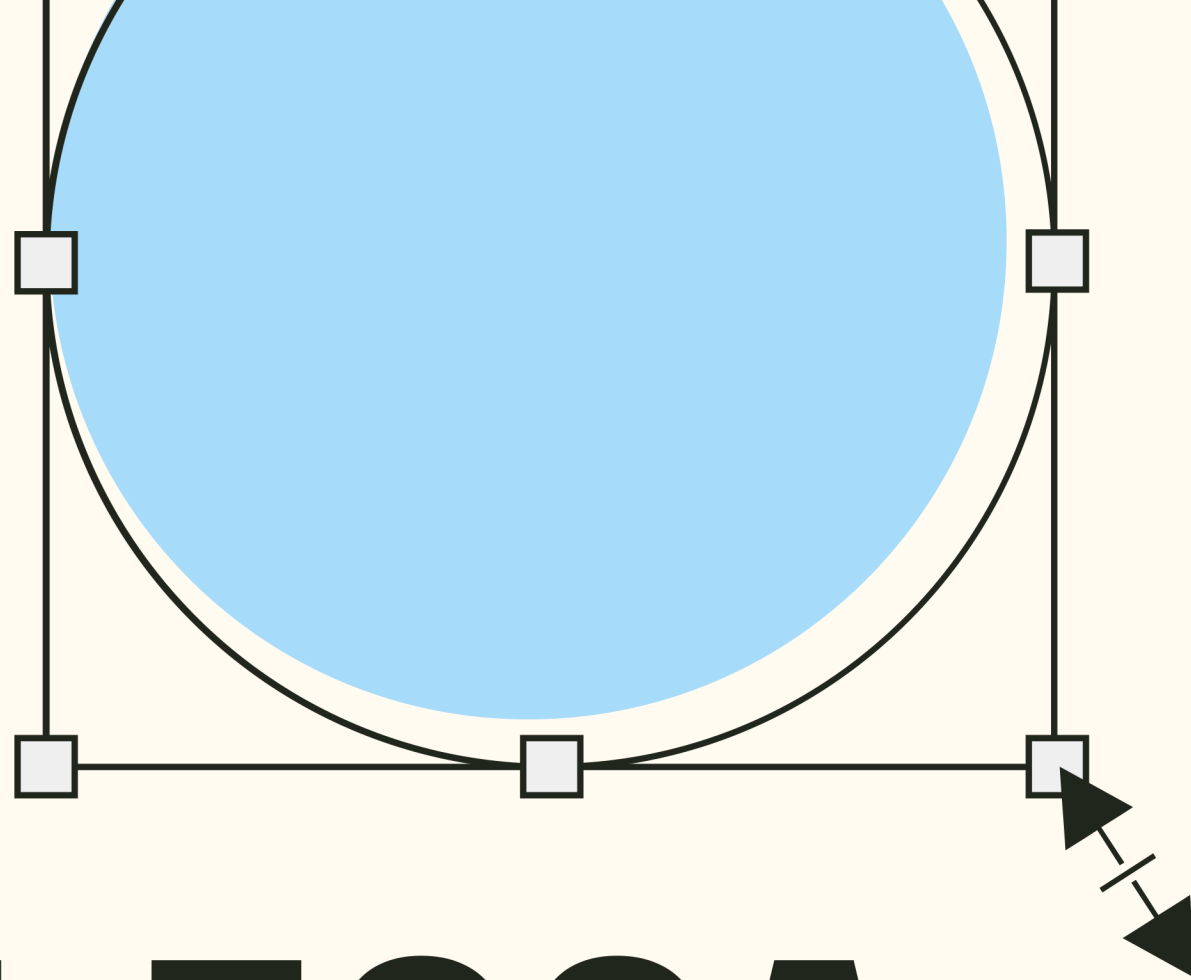
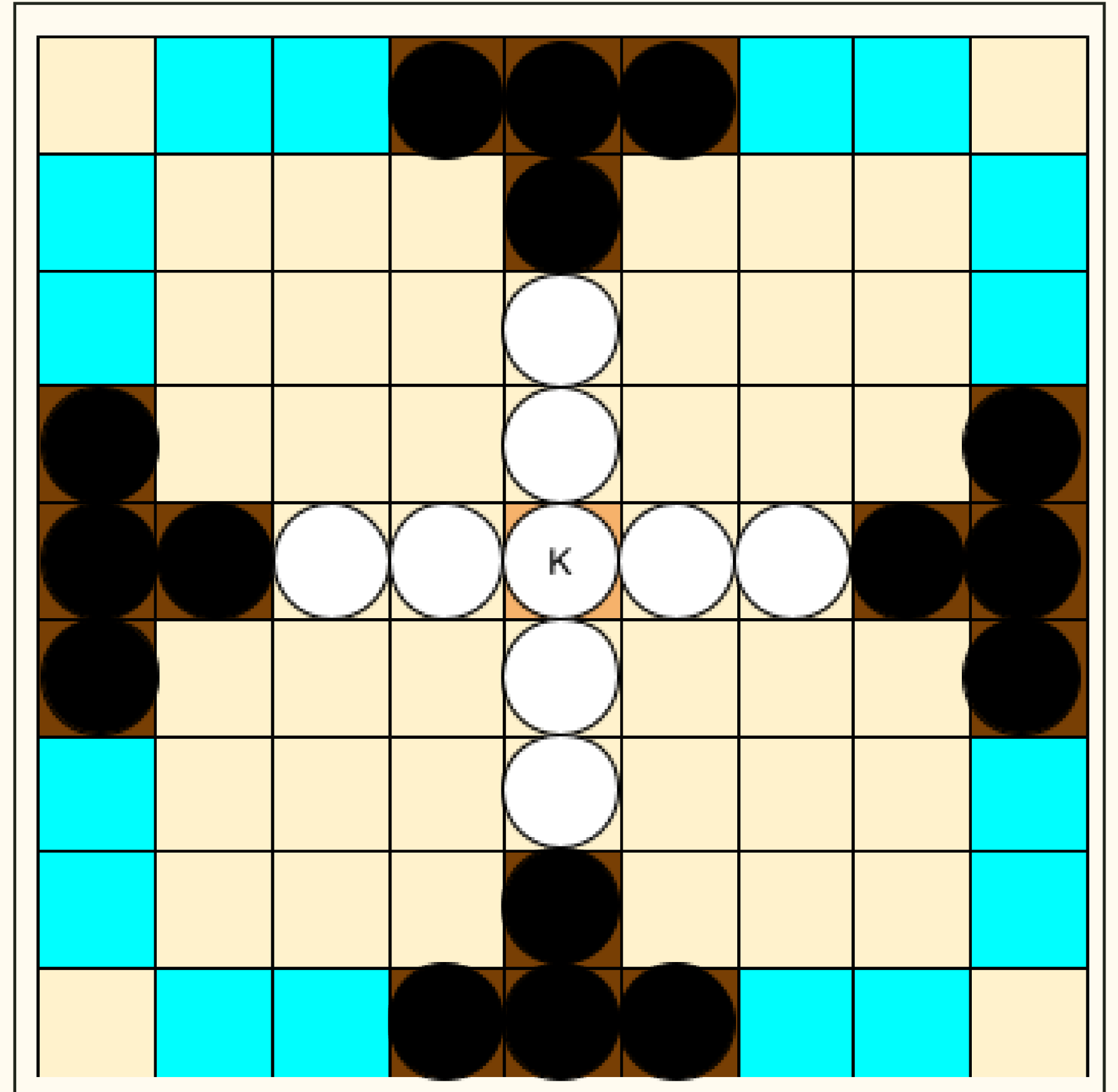


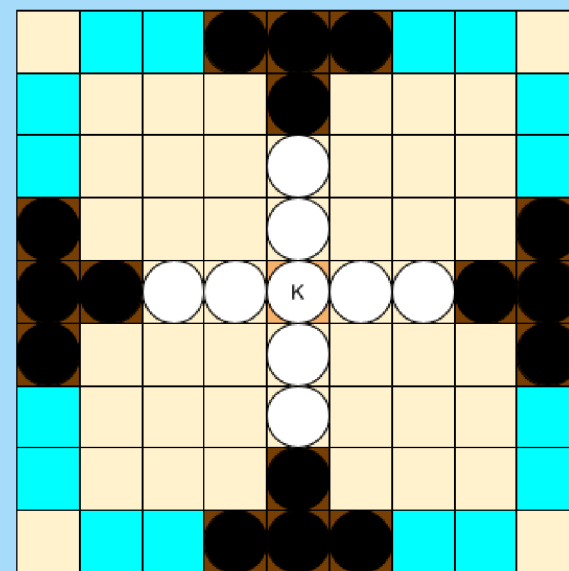


MELISSA
I LOVE
TABLUT

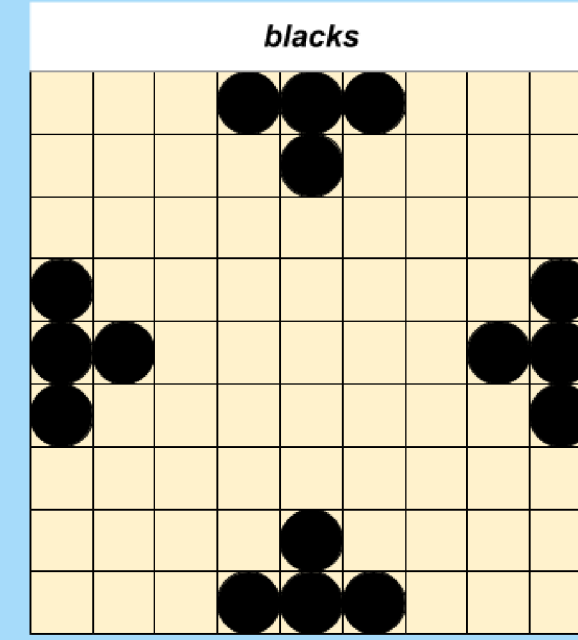
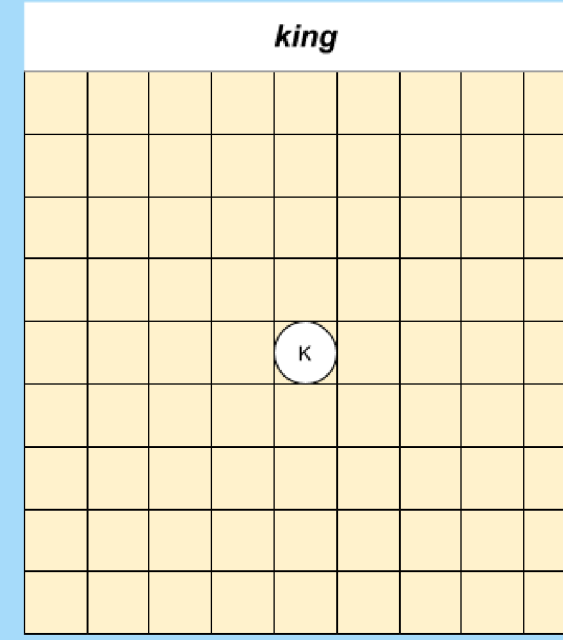
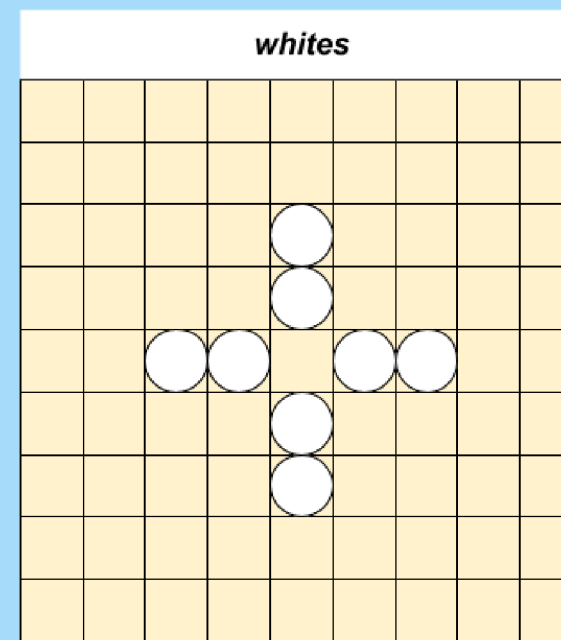


State Representation

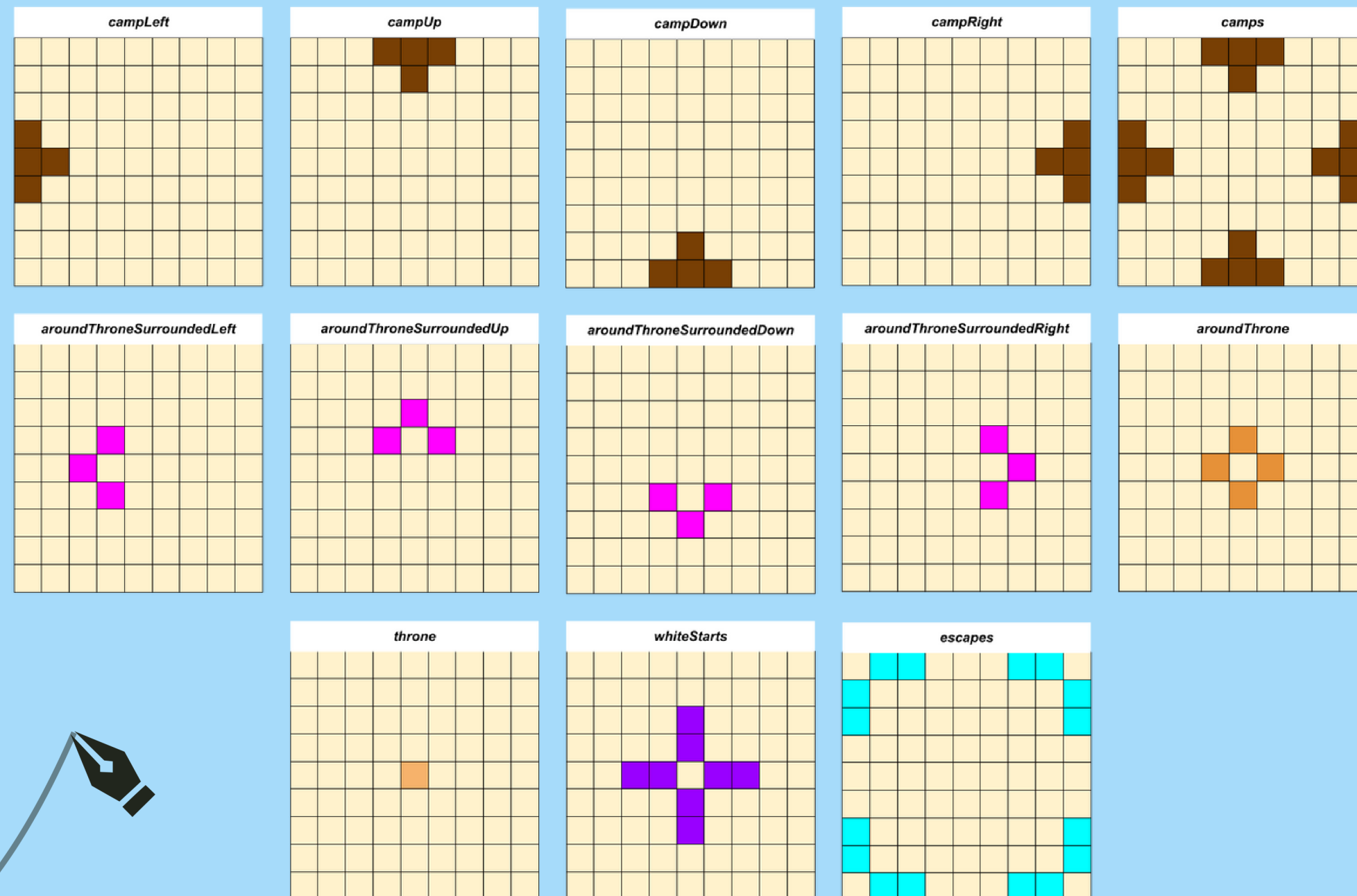
In order for our player to be as efficient as possible, we decided to encode the state with java Bitmaps.

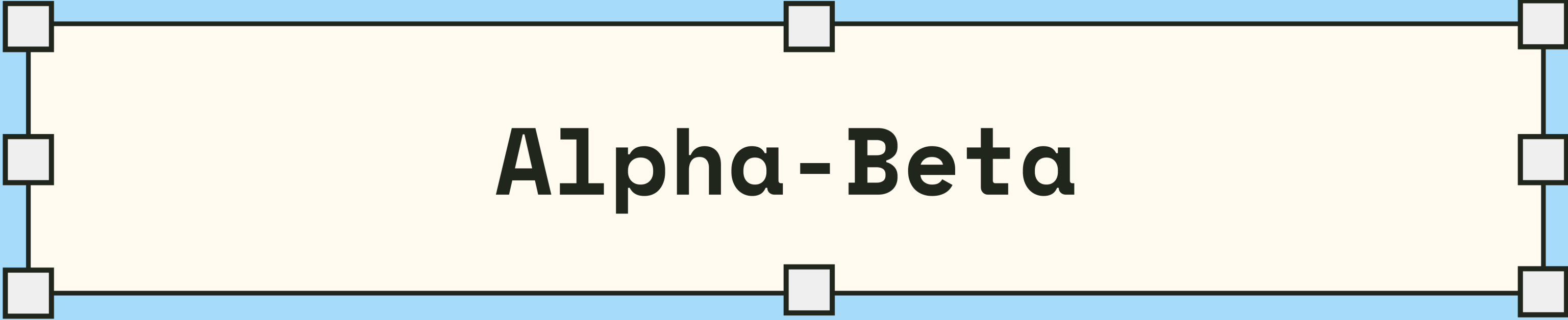


=



State Representation

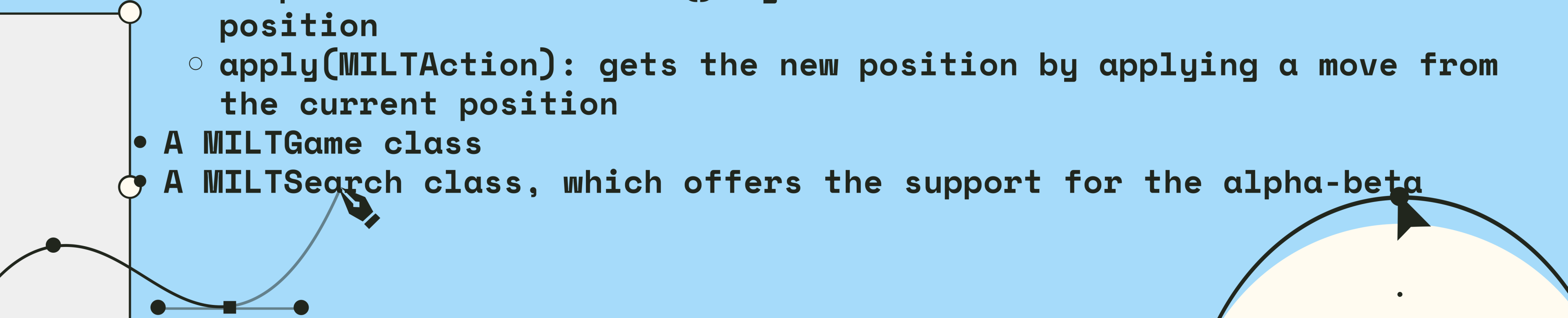




Alpha-Beta

For the best-move search we used the aima implementation of the iterative deepening alpha-beta algorithm.

We created:

- A MILTAction class, representing a move
 - A MILTState class, which represents a Tablut position and offers the following methods:
 - computeAvailableActions(): gets available moves from current position
 - apply(MILTAction): gets the new position by applying a move from the current position
 - A MILTGame class
 - A MILTSearch class, which offers the support for the alpha-beta
- 

Heuristics

For the heuristics we adopted the Visitor pattern, creating two MILTEvaluator classes, one for the black and one for the white, acting as visitors of MILTState. Our heuristics considers:

- the difference between black and white pawns
- the number of threatened white and black pawns
- king threats
- the number of potentially available escapes for the king
- the number of effective available escapes for the king
- the number of empty 'ways', where a way is one of the squares where the white pawns are initially positioned

Each component of the heuristic will be multiplied by its corresponding weight

King positions

